Assignment of the course

Numerical Algorithms / Numerical Calculus

Second semester 2024/2025

Assignment 1: Nonlinear solvers

The solution of this assignment should be uploaded in Moodle as a single compressed folder containing all the implemented code (which should be reasonably readable), and a short report containing the plots and the comments required in the following. For the implementation, you can follow the draft code that is provided.

Task 1 (One-dimensional nonlinear solvers) .....

Consider a function $f : \mathbb{R} \to \mathbb{R}$, which can be implemented in Python as a function

```
def f(x):
    # y = ...
    return y
```

or as an anonymous function

```
f = lambda x: # Function definition
```

The derivative f', when required, is similarly implemented by a function `df`. Define moreover a maximal number of iterations $k_{\max} \in \mathbb{N}$, a residual tolerance $\varepsilon_{tol} > 0$, and an increment tolerance $\varepsilon_{inc} > 0$.

Implement the following algorithms. Each of them should return a NumPy array `x` containing all the iterates $x_0, x_1, \ldots$, and an array `fx` containing the values $|f(x_k)|$, $k = 0, 1, \ldots$.

(a) The bisection method, that takes as input the function f, an interval $[a, b]$, and the tolerance parameters defined above. The method should be implemented by a function

```
def bisection(f, a, b, k_max, eps_res, eps_tol):
    # Function definition
    return x, fx
```

(b) The Newton method, that takes as input the function f, its derivative f', an initial guess x_0, and the tolerance parameters defined above. The method should be implemented by a function

```
def newton(f, df, x0, k_max, eps_res, eps_tol):
    # Function definition
    return x, fx
```

(c) The simplified Newton method, that takes as input the function f, an initial guess x_0, the value of the derivative $f'(x_0)$, and the tolerance parameters defined above. The method should be implemented by a function

```
def simplified_newton(f, df0, x0, k_max, eps_res, eps_tol):
    # Function definition
    return x, fx
```

You can use the code sketch provided in `utils.py`.

Task 2 (Testing of the solvers) .....

Consider the function $f : \mathbb{R} \to \mathbb{R}$ defined by

$$f(x) := x^3 - x - 1,$$

which has a unique real zero $\bar{x} \in \mathbb{R}$. For this task we set $\varepsilon_{tol} = \varepsilon_{inc} = 10^{-10}$ and $k_{\max} = 100$.

- Plot the function f in $[-3, 3]$ and find suitable values $a, b \in \mathbb{R}$ such that and the bisection method with initial interval $[a, b]$ converges to $\bar{x}$. Explain your choice of a, b.
- Compute f'. We want to run the Newton method with $x_0 \in [0, \bar{x}]$: What is the smallest value of $x_0 \in [0, \bar{x}]$ such that the method converges? Explain your choice.
- Run the simplified Newton method with the same x_0 found in the previous point. Is the method converging? If not, try to find experimentally the minimal value of $\tilde{x}_0 \in [x_0, \bar{x}]$ such that the method converges. Discuss the results.

For each point, report the plots of the residuals $|f(x_k)|$ and the iterations $x_0, x_1, \dots$ using the code provided in `01-test_solvers.py`.

Task 3 (Nonlinear least-squares) .....

We are modelling a monitoring system where a set of $n \in \mathbb{N}$ sources emit a pollutant with concentrations $x := (x_1, \dots, x_n) \in \mathbb{R}^n$, and a set of $m \in \mathbb{N}$ measuring stations measure concentrations $y := (y_1, \dots, y_m) \in \mathbb{R}^m$. The location of these sources and stations are known, and stored into the matrices `X`, `Y` in `utils.py`. They are used for visualization only.
We assume that the relation between the emissions and the measures is modeled by a function $y = p(x)$, where $p : \mathbb{R}^n \to \mathbb{R}^m$ has components $p(x) := (p_1(x), \dots, p_m(x))$. Each component $p_i(x)$ accounts for a superposition of all sources with a saturation effect. More precisely, we assume that

$$p_i(x) := \sum_{j=1}^{n} A_{ij} \left(\frac{x_j}{1 + \beta x_j} \right), \quad i = 1, \dots, m,$$

where $A_{ij} \in \mathbb{R}$ is an influence coefficient of source j on sensor i, and $\beta > 0$ is a given saturation parameter. For $\beta = 0$ the system is linear, while it is increasingly non linear for larger values of β. The matrix `A` and the function `p` are provided in `utils.py`, while we will analyze different values of $\beta > 0$. We show the sensors-sources distribution, and two examples of x, y values in Figure 1.

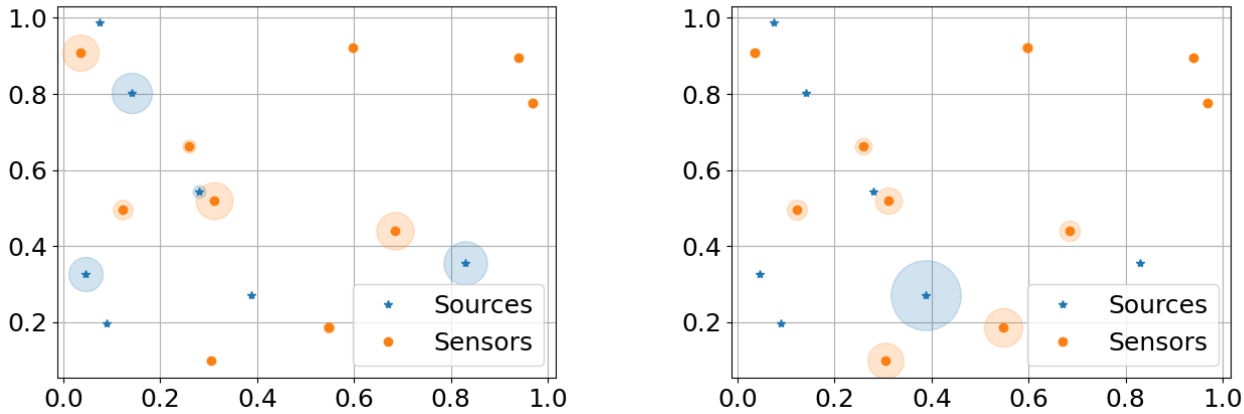

Figure 1: Distribution of the sensors (orange dots) and the sources (blue stars). The size of the shaded circle around each point is proportional to the value of x_i or y_i. The two examples are obtained with a random x (left) and with $x = (0, 0, 1, 0, 0, 0, 0)$ (right).

We are given a vector $\bar{y} = (\bar{y}_1, \dots, \bar{y}_m) \in \mathbb{R}^m$ of measures collected by the sensors, and our goal is to estimate the unknown values $\bar{x} = (\bar{x}_1, \dots, \bar{x}_n) \in \mathbb{R}^n$ emitted by the sources. We estimate these values by minimizing the function $F : \mathbb{R}^n \to \mathbb{R}$ defined by

$$F(x) := \sum_{i=1}^{m} (\bar{y}_i - p_i(x))^2.$$

Our estimation $\hat{x}$ is thus

$$\hat{x} := \arg\min_{x \in \mathbb{R}^n} F(x),$$

and we find this minimum by finding a stationary point of F, i.e., setting $f : \mathbb{R}^n \to \mathbb{R}^n$ with $f(x) := \nabla F(x)$, we look for $\hat{x} \in \mathbb{R}^n$ such that $f(\hat{x}) = 0$. By direct computation it is possible to find that

$$f(x) = -2 \cdot \frac{A^T(\bar{y} - p(x))}{(1 + \beta x)^2},$$

where the division means division of two vectors component by component. We do not compute the Jacobian of f in general because it is too complicated. Instead, we just compute it for $\beta = 0$ (the linear setting) and in this case we get

$$Df(x) = 2 \cdot A^T A. \tag{1}$$

(a) Implement the function $f(x)$ as a function

```
def f(x, y, A, beta):
    # val = ...
    return val
```

(b) Update the implementation of the simplified Newton method of Task 1 to work with functions $f : \mathbb{R}^n \to \mathbb{R}^n$. For the iteration, instead of using the Jacobian computed at the initial point, the method should use the Jacobian (1). The function should be implemented as

```
def quasi_newton_nd(f, df0, x0, k_max, eps_res, eps_inc):
    # Function definition
    return x, fx
```

where `df0` is the matrix (1).

(c) Use this method to find $\hat{x}$ for $\hat{x} = (0, 0, 1, 0, 0, 0, 0)$, and compute the error $\|\bar{x} - \hat{x}\|$. Plot this error as a function of β for $\beta = 0, 0.01, \ldots, 0.1$. Discuss the results.

A sketch of the solution and some visualization code is provided in `01-parameter_estimation.py`.